

Lecture 5:

Data classification (III)

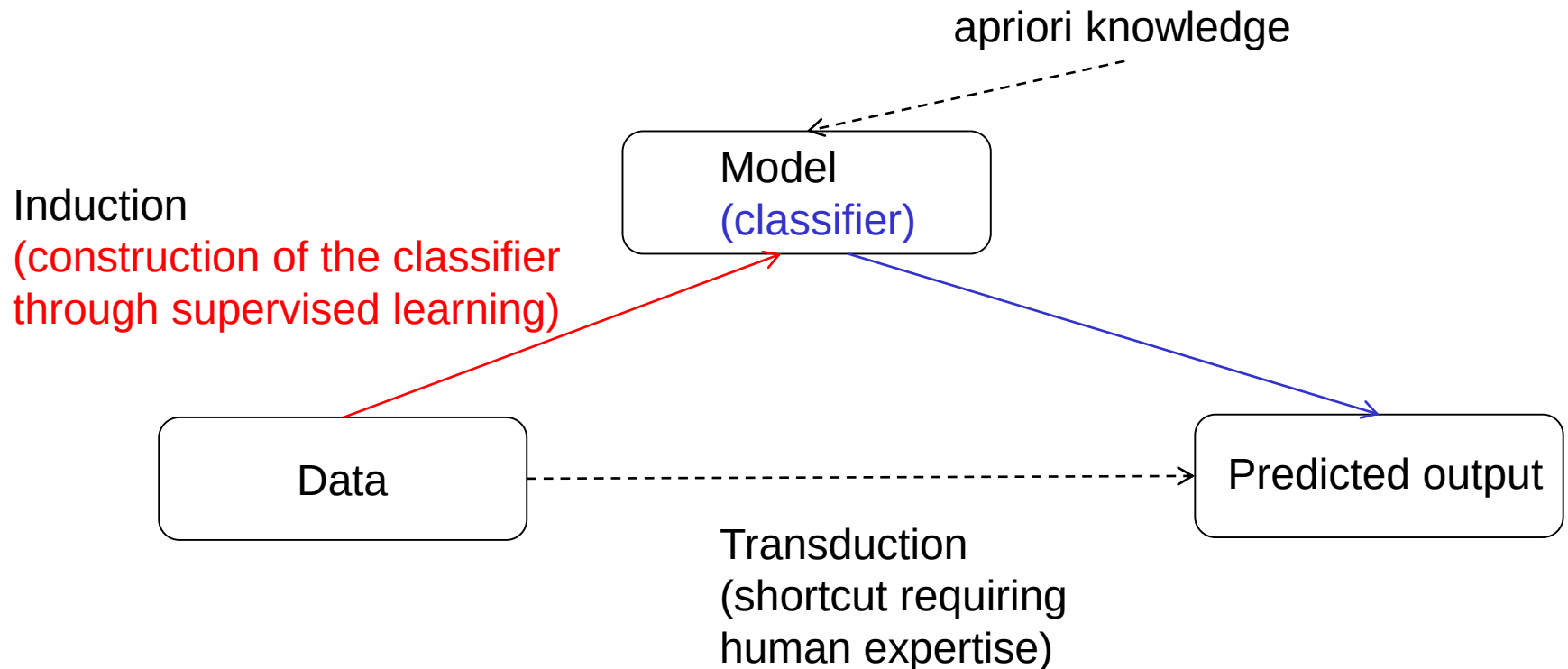
(k Nearest Neighbours and Naïve Bayes classifiers)

Reminder: classification models

Learning/ induction/ inference = construct a model starting from data (and some apriori knowledge specific to the domain)

Different ways of using data, models and knowledge:

induction vs deduction vs transduction



Reminder: classification models

A classification model is a “mapping” between attributes and class labels

Example of classification models:

- Decision trees
- Classification rules
- Prototypes (exemplars)
- Probabilistic models
- Function-based models (Neural networks, Support Vector Machines)

The classification model should be:

- Accurate:
 - Identify the right class
- Compact / comprehensible
 - Easy to be understood/ interpreted by the user (it is preferable to not be a black box)
- Efficient in the
 - Learning/training step
 - Classification step

Outline

- Instance (exemplar) based classification: **kNearest Neighbours**
- Probabilistic classification: **Naïve Bayes**

Instance based classifiers

Main idea: similar instances have similar class labels (analogy-based reasoning)

- The classification model consists of the examples from the training dataset
 - The training process consists only of **storing the training examples**
- The classification of a new data instance consists of:
 - Compute the **similarities** (or dissimilarities) between the new data instance and the examples in the training set and find the **most similar examples**
 - Choose the **most frequent class** from the set of the similar examples

Instance based classifiers

Main idea: similar instances have similar class labels (analogy-based reasoning)

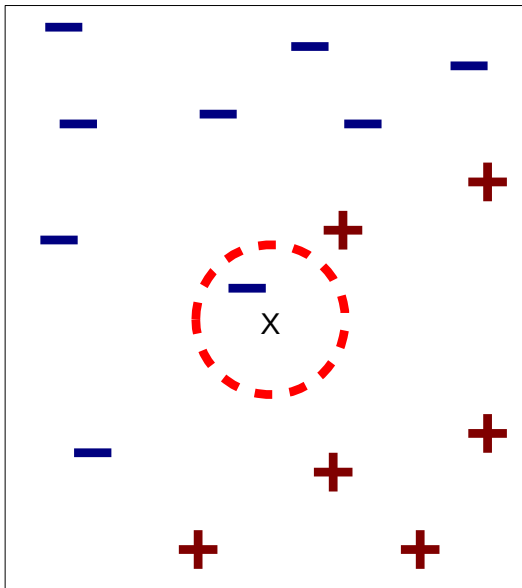
Remarks:

- Such classifiers are usually called “**lazy**” because the training step does not involve any effort (the entire computational effort is postponed for the classification step)
- The most popular lazy classifiers are those based on the **Nearest Neighbors** principle
- **Applications**
 - Recommender systems
 - Medical diagnosis

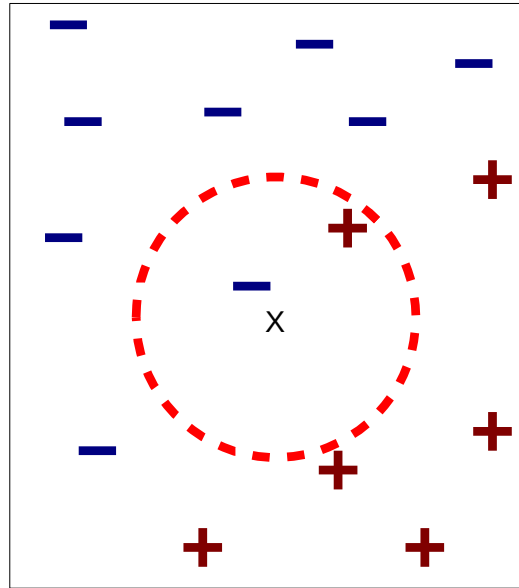
Instance based classifiers

kNN – k Nearest Neighbour

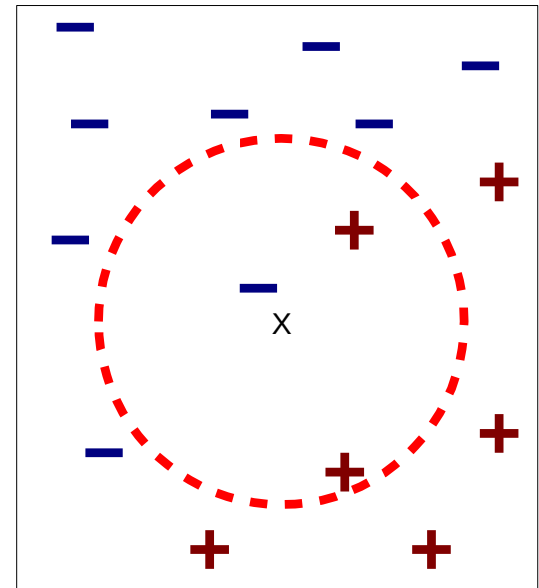
- For each data to be classified:
 - Find the **closest (most similar) k examples** from the training dataset
 - Identify the most frequent class (majority class)



(a) 1-nearest neighbor



(b) 2-nearest neighbor



(c) 3-nearest neighbor

[Tan, Steinbach, Kumar; Introduction to Data Mining, slides, 2004]

Instance based classifiers

kNN – k Nearest Neighbour

- For each data to be classified:
 - Find the **closest (most similar) k examples** from the training dataset (set of exemplars)
 - Identify the **most frequent class**

The performance of kNN classifiers depend on:

- The **similarity/ dissimilarity (distance) measure**
 - It depends on the attribute types and on the particularities of the problem
- The **value of k (the number of neighbors)**
 - Simplest case: $k=1$ (not very good in the case of noisy data)

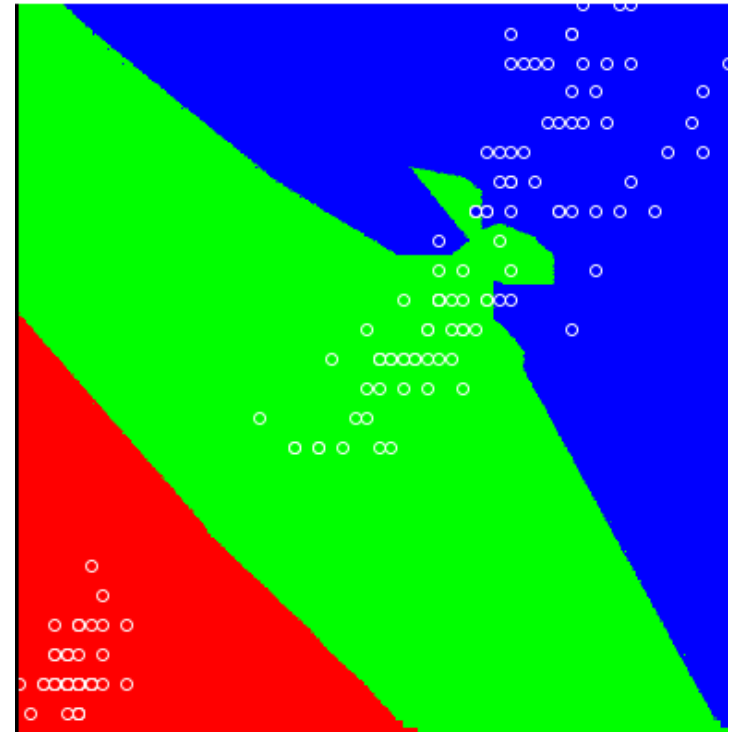
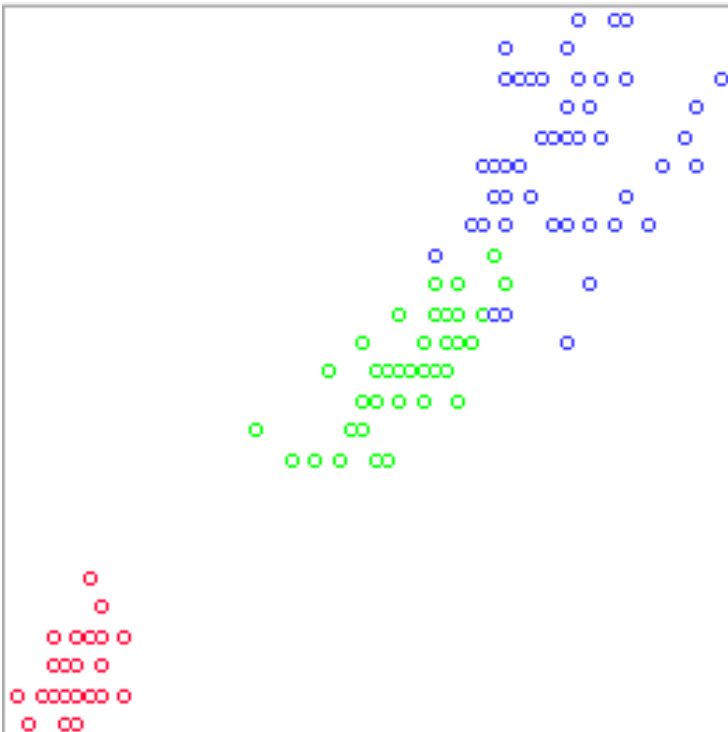
Remark: kNN induces a partition of the data space in regions; the regions are not explicitly computed but are implicitly determined by the similarity measure (and the value of k)

Instance based classifiers

1NN = Nearest Neighbor with one closest neighbor (based on the normalized Euclidean distance)

Illustration of the geometric boundaries. Dataset: iris2D (“petal length” and “petal width”).

Plot: Weka->Visualization->BoundaryVisualizer



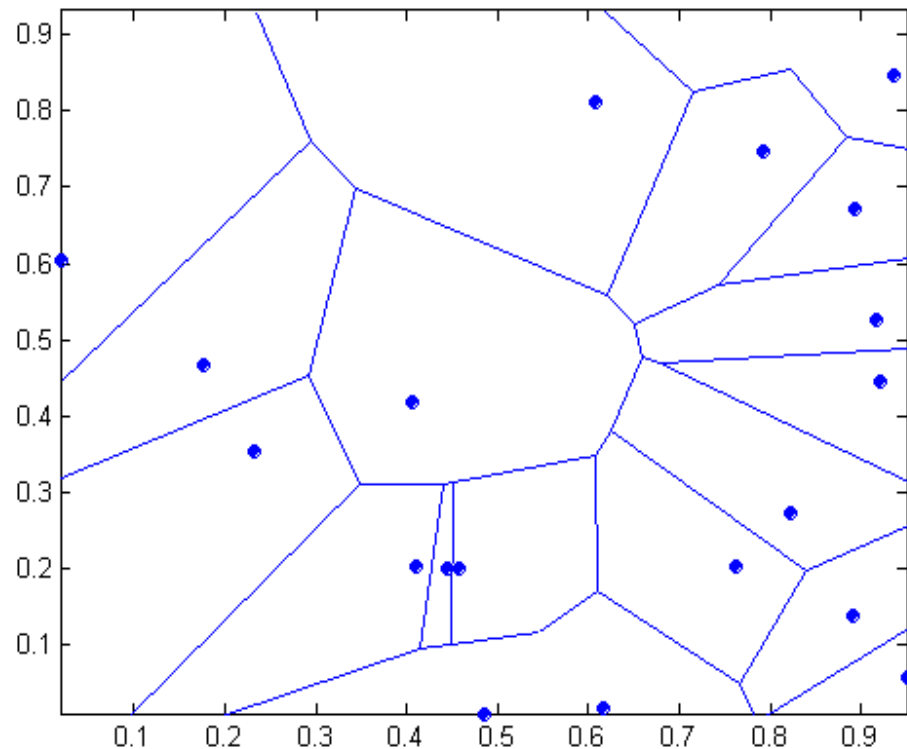
Instance based classifiers

1NN = Nearest Neighbor with one closest neighbor (based on the normalized Euclidean distance)

1NN induces a partitioning of the data space (e.g. in 2D this corresponds to a Voronoi diagram)

Remark:

- Each instance in the training dataset has associated a region containing all data which are in the neighborhood of that training instance
- Voronoi diagrams have applications in:
 - Geography (GIS): territory partitioning
 - Robotics: path planning
 - Computer graphics: procedural textures
 - Biology: cell structures modelling



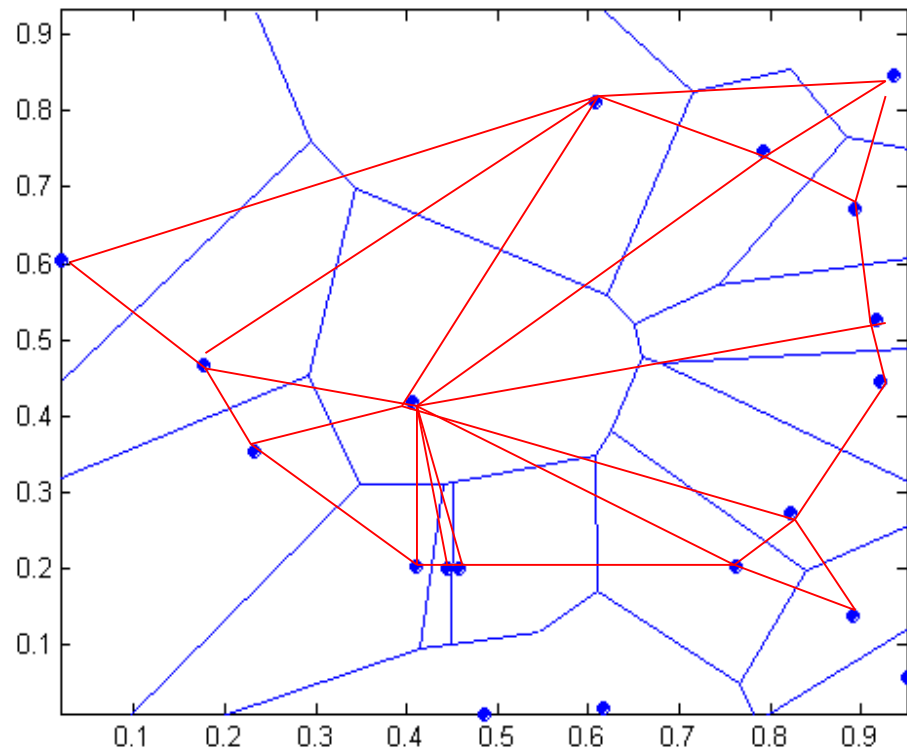
[Source: Tan, Steinbach, Kumar; Introduction to Data Mining, slides, 2004] 10

Instance based classifiers

Voronoi diagram → Delaunay triangularization

Other applications:

- Graphics & Game development:
 - Surface reconstruction
 - Mesh generation for rendering
 - Creating realistic landscapes
- Engineering:
 - Meshes for finite elements modeling
 - Wireless networks
 - Autonomous driving (building maps from LiDAR point clouds)



[Source: Tan, Steinbach, Kumar; Introduction to Data Mining, slides, 2004]

Similarity/ dissimilarity measures

Let us consider two entities (e.g. data vectors, time series, texts etc.) A and B

- A **similarity measure** $S(A,B)$ is a number which is higher if A and B are more similar
- A **dissimilarity measure** $D(A,B)$ is a number which is higher if A and B are less similar (or more different)

The choice of an appropriate measure depends on:

- The type of attributes
- The number of attributes
- The distribution of the data
- The particularities of the problems (domain-based knowledge)

Similarity/ dissimilarity measures

Numerical attributes

Most popular dissimilarity measures:

- Euclidean distance
- Manhattan distance

Remarks:

- The Euclidean distance is invariant with respect to rotations
- If not all attributes have the same importance then weights should be included in the distance (e.g. $w_i(a_i - b_i)^2$ instead of $(a_i - b_i)^2$)

$$d_p(A, B) = \sqrt[p]{\sum_{i=1}^n (a_i - b_i)^p} \quad (\text{Minkowski, } L_p)$$

$$d_E(A, B) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \quad (\text{Euclidean, } p = 2)$$

$$d_M(A, B) = \sum_{i=1}^n |a_i - b_i| \quad (\text{Manhattan, } p = 1)$$

$$d_\infty(A, B) = \max_{i=1, \dots, n} |a_i - b_i| \quad (p = \infty)$$

Remark: The weights can be estimated by using preprocessing methods; instead of using explicit weights the data can be scaled/ standardized

Similarity/ dissimilarity measures

Numerical attributes

$$d_p(A, B) = \sqrt[p]{\sum_{i=1}^n (a_i - b_i)^p} \quad (\text{Minkowski, } L_p)$$

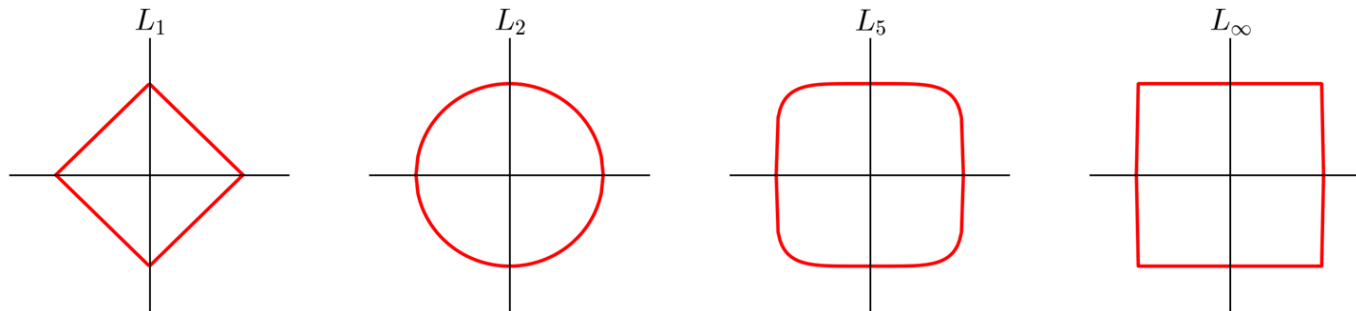
Differences between dissimilarity measures

$$d_E(A, B) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \quad (\text{Euclidean, } p = 2)$$

$$d_M(A, B) = \sum_{i=1}^n |a_i - b_i| \quad (\text{Manhattan, } p = 1)$$

$$d_\infty(A, B) = \max_{i=1, \dots, n} |a_i - b_i| \quad (p = \infty)$$

Set of points placed at the same distance with respect to the origin of the system of coordinates:



Source: Lecture 19 - Data Science, Steven Skiena, Stony Brook University

Similarity/ dissimilarity measures

Practical issues – dimensionality curse:

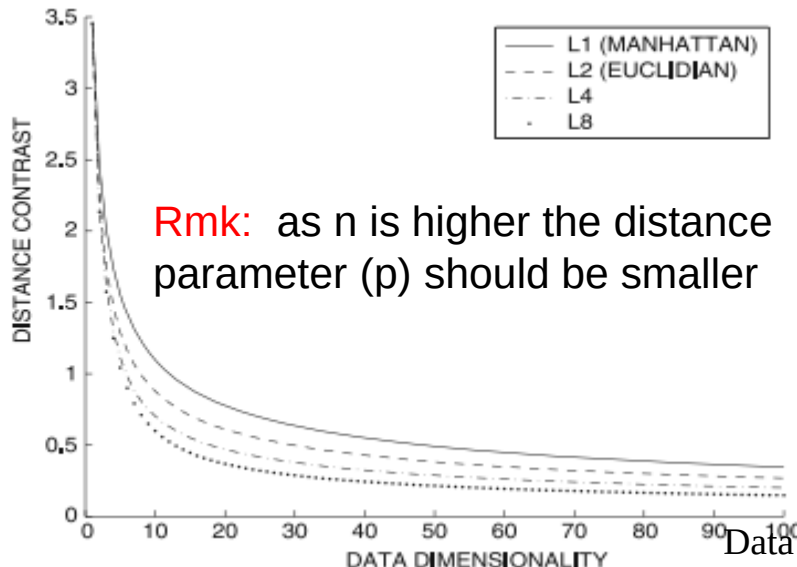
- These distances lose their discriminative power as the size (n) increases => for high-dimensional data the distance-based classifiers may be qualitatively ineffective

Remark: it might be useful to reduce the dimensionality of the data before using kNN (e.g. by using PCA or another technique, like tSNE, UMAP)

$$\text{Distance contrast} : \frac{d_{\max} - d_{\min}}{\sigma}$$

$d_{\max}, d_{\min}$ = largest and smallest distance

σ = standard deviation of distances

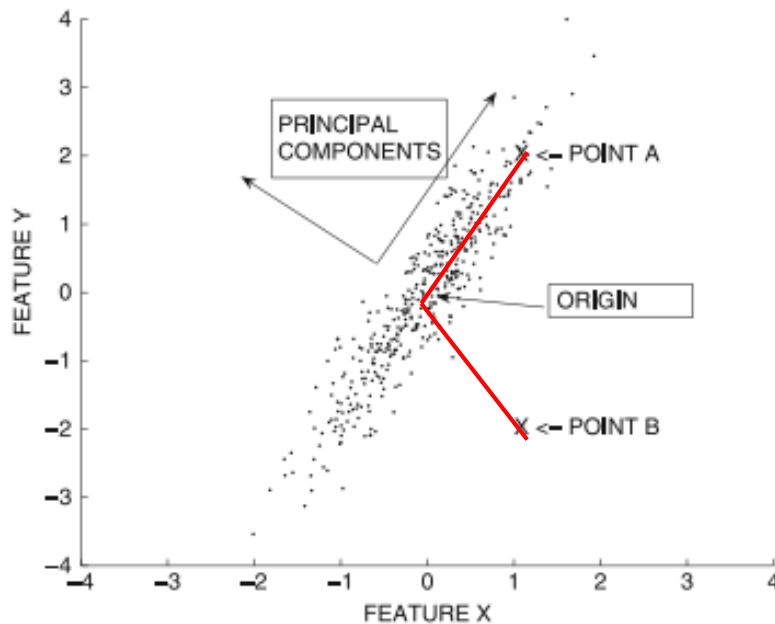


[Source: Aggarwal, Data Mining Textbook, 2015]

Similarity/ dissimilarity measures

Practical issues – impact of data distribution

Q: Which is closer to the origin? Point A or point B?



[Source: Aggarwal, Data Mining Textbook, 2015]

Similarity/ dissimilarity measures

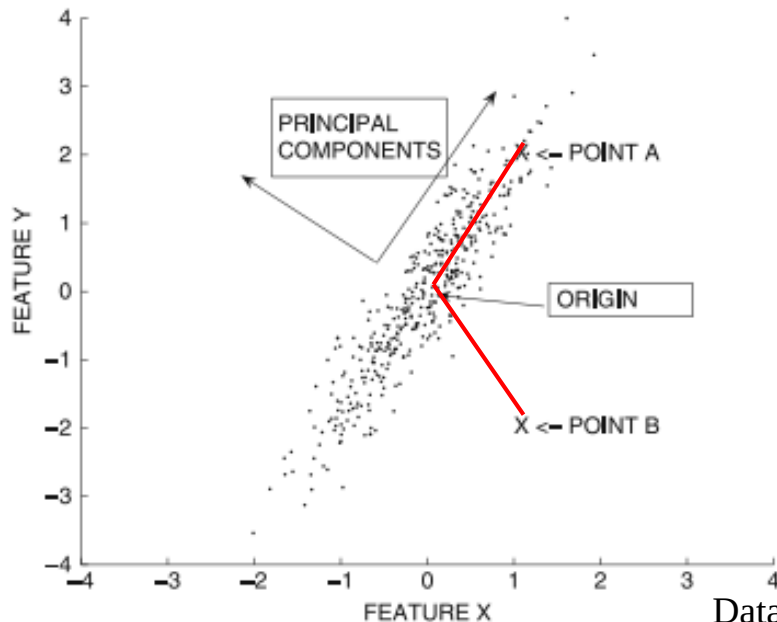
Practical issues – impact of data distribution

Q: Which is closer to the origin? Point A or point B?

A: The Euclidean distances $d(O,A)$ and $d(O,B)$ are equal. However taking into account the data distribution, A is more in agreement with the data distribution than B, thus it should be considered that A is “closer” to O than B

New question: how can be quantified the data distribution in the distance?

Answer: use the Mahalanobis distance



Mahalanobis distance

$$d_{Mah}(A, B) = \sqrt{(A - B)^T C^{-1} (A - B)}$$

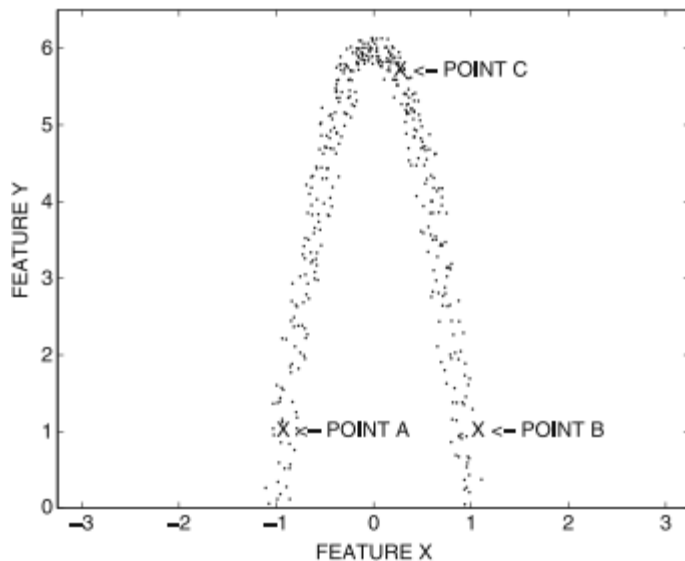
C^{-1} = inverse of the covariance matrix

[Source: Aggarwal, Data Mining Textbook, 2015]

Similarity/ dissimilarity measures

Practical issues – impact of data distribution

Q: Is the distance between A and B smaller than the distance between B and C?



[Source: Aggarwal, Data Mining Textbook, 2015]

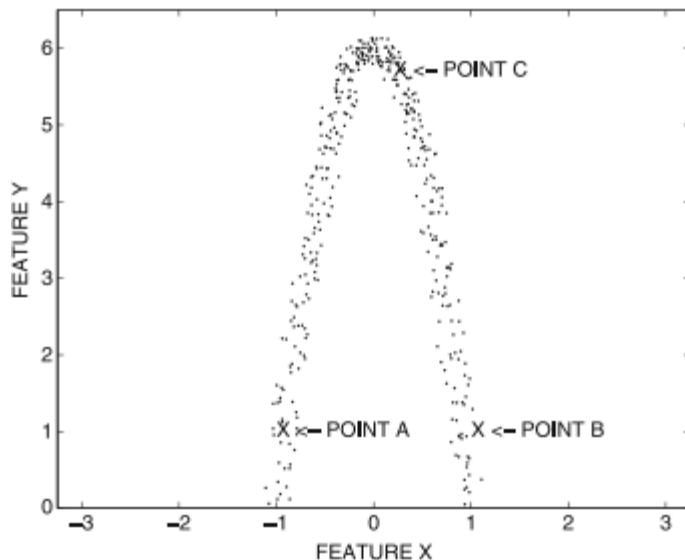
Similarity/ dissimilarity measures

Practical issues – impact of data distribution

Q: Is the distance between A and B smaller than the distance between B and C?

A: Yes, if we ignore the data distribution and we use the Euclidean distance

However, the data distribution provides a context for the problem to be solved and in this context $d(A,B) > d(B,C)$



Geodesic distance:

- Construct a graph by using the points as nodes and by defining edges between nearest neighbors (each point is connected to its first k nearest neighbors)
- Compute the distance between two points as the shortest path in the graph.

[Source: Aggarwal, Data Mining Textbook, 2015]

Similarity/ dissimilarity measures

Numerical attributes – similarity measure

- Cosine measure: $\text{sim}(A,B)=A^T B/(||A|| \ ||B||)$ (scalar product between A and B divided by the product of the norms)

Remark:

- In the case of normalized data vectors ($||A||=||B||=1$) the similarity between the vectors is maximal when the Euclidean distance between them is minimal:

$$\begin{aligned} d_E^2(A, B) &= (A - B)^T (A - B) = A^T A - 2A^T B + B^T B = \\ &= 2(1 - A^T B) = 2(1 - \text{sim}(A, B)) \end{aligned}$$

Similarity/ dissimilarity measures

Nominal attributes

Approach 1: Transformation of nominal attributes in numerical ones (by binarization) and use similarity/dissimilarity measures for **binary vectors**:

- Dissimilarity: **Hamming distance** = Manhattan distance: $d_H(A,B)=d_M(A,B)$

- Similarity: **Jaccard coefficient**
$$J(A, B) = \frac{\sum_{i=1}^n a_i b_i}{\sum_{i=1}^n (a_i + b_i - a_i b_i)} = \frac{\text{card}(S_A \cap S_B)}{\text{card}(S_A \cup S_B)}$$

- Similarity: **Dice coefficient**
$$D(A, B) = \frac{2 \sum_{i=1}^n a_i b_i}{\sum_{i=1}^n (a_i + b_i)} = \frac{2 \text{card}(S_A \cap S_B)}{\text{card}(S_A) + \text{card}(S_B)}$$

Remark: S_A and S_B are the subsets of the global set of n attributes which correspond to the **membership vectors A and B**, respectively

Similarity/ dissimilarity measures

Nominal attributes

Approach 2: Use local similarity measures (between attribute values)

$$S(A, B) = \sum_{i=1}^n S(a_i, b_i)$$
$$S(a_i, b_i) = \begin{cases} 1 & \text{if } a_i = b_i \\ 0 & \text{if } a_i \neq b_i \end{cases}$$

Remark: the similarities which are unusual should be considered more significant than those which are frequent

$$S(a_i, b_i) = \begin{cases} 1 / f^2(a_i) & \text{if } a_i = b_i \\ 0 & \text{if } a_i \neq b_i \end{cases}$$

$f(a_i)$ = frequency of the value a_i in the dataset
(for i - th attribute)

Similarity/ dissimilarity measures

Mixed attributes: the measures corresponding to numerical and nominal attributes should be aggregated (using specific weights)

$$S(A, B) = \lambda S_{\text{numerical}}(A, B) + (1 - \lambda) S_{\text{nominal}}(A, B)$$

Other types of data:

- Strings (e.g. text or biological sequences) - use of the **edit distance** (or similarity measures based on **sequences alignment**, e.g. Dynamic Time Warping)
- Concepts (e.g. nodes in an ontology) – **path-based distances in trees/ graphs**
- Graphs (e.g. social or biological networks) – amount of **similar small structures** (patterns)

Remark: non-numerical data can be transformed into numerical vectors by **embedding** using machine learning models (e.g. BERT versions for text, GNN variants for graphs etc)

Nearest Neighbour: choice of k

The performance of kNN is sensitive to the number (k) of neighbours

Extreme cases:

- $k=1$ - the classifier is not robust (errors in the dataset could mislead the classification result)
- $k=N$ - it is equivalent to ZeroR being based only on the distribution of the data in classes (the sensitivity of the classifier to the underlying data is lost)

How to choose k?

- Trial-and-error approach: try different values and choose the one providing the best performance (cross-validation)

Nearest Neighbour: computational cost

In a naïve implementation, the classification step requires the computation of $N-1$ distances (or similarities) for a dataset with N elements (n -dimensional vectors) and the selection of the smallest k distances → $O(Nn+kN)$

When N is large this could be costly (as it should be done for each new data that has to be classified)

Possible approaches:

- Create **indexing data structures** allowing to find the k nearest neighbours in an efficient way (kd-trees, locality sensitive hashing = similar instances are placed in the same bucket)
- Identify small **clusters** in the data set and replace each such cluster with a single prototype
- Use **instance (prototype) selection** = select the most representative instances from the dataset (not easy to decide which ones are representative)

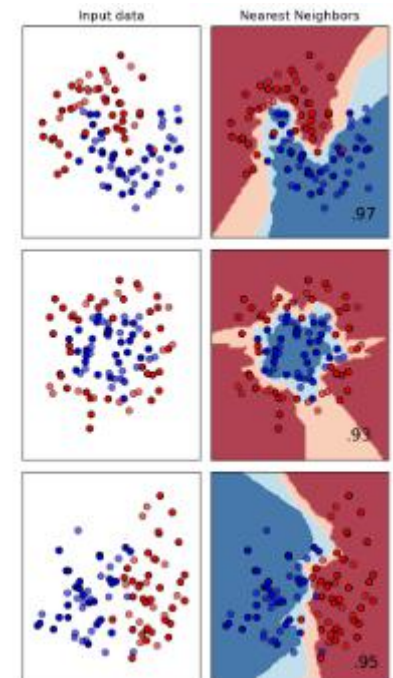
Summary - kNN

- Advantages

- Efficient training - lazy training (however it might require advanced data structures to store the exemplars)
- Naturally adapted for incremental learning (e.g. data streaming)
- Work well with small datasets
- Easy to use for multiple classification
- Can be easily adapted for regression
- Can be used for multi-modal data – can capture rather complex decision surfaces

- Disadvantages

- Not suitable for large datasets (memory intensive)
- Sensitive to the choice of the similarity measure and to the choice of the number of neighbours
- Sensitive to imbalanced datasets



Probabilistic Classification

Idea: construct a model which estimates the probability of a data instance to belong to a given class

Aim: estimate $P(C_k|D_i)$ = probability that the class of data instance D_i is C_k

Remark: $P(B|A)$ is a conditional probability = probability of event B given that (by assumption, presumption, assertion or evidence) event A has occurred

Reminder on probability theory:

$$\text{Conditional probability : } P(B | A) = \frac{P(A, B)}{P(A)}$$

$P(A, B)$ = probability that both events A and B occurred

$$\text{Bayes rule : } P(B | A) = \frac{P(A | B)P(B)}{P(A)}$$

Bayes rule is useful to estimate the **posterior probability** $P(B|A)$ when the **prior probability** $P(B)$, the **likelihood** $P(A|B)$, and the **evidence** $P(A)$ can be easily estimated.

Probabilistic Classification

Example: Let us suppose that we are interested to estimate the probability that a patient having some symptom S has the illness T

- We want to estimate $P(T|S)$
- Let us suppose that we know:
 - $P(T)$ – estimated based on population studies (how frequent is the illness) – this is the **prior probability**
 - $P(S|T)$ – estimated based on prior medical knowledge (how often is the symptom present in the case of illness T) - this is **likelihood**
 - $P(S)$ – the probability of observing the symptom – this is **evidence**
 - $P(T|S)=P(S|T)P(T)/P(S)$ (if $P(S)=1$ then $P(T|S)=P(S|T)P(T)$)
- What about the case when there is not only one symptom S , but a list of symptoms $S_1, S_2, \dots, S_n$?

Probabilistic Classification

Example: Let us suppose that we are interested to estimate the probability that a patient having some symptoms $S_1, S_2, \dots, S_n$ has the illness T

- In this case one have to estimate $P(T | S_1, S_2, \dots, S_n)$
- Based on the Bayes rule:
 - $P(T | S_1, S_2, \dots, S_n) = P(S_1, S_2, \dots, S_n | T) P(T) / P(S_1, S_2, \dots, S_n)$
- **Simplifying assumption:** the symptoms $(S_1, S_2, \dots, S_n)$ are **independent events** given the illness T ; this is known as **class - conditional independence** property (this is not always true but in many practical situations this assumption can be accepted – e.g. spam filtering)
- Considering that $P(S_1, S_2, \dots, S_n)$ is constant given the input data, to decide the class it is enough to use only the numerator

$$P(T | S_1, S_2, \dots, S_n) \approx P(S_1 | T) P(S_2 | T) \dots P(S_n | T) P(T)$$

Naïve Bayes Classifier

Classification problem:

- For a data instance $D_i=(a_{i1},a_{i2},\dots,a_{in})$ find the class to which it belongs

Main idea

- Estimate $P(C_k|D_i)=P(a_{i1},a_{i2},\dots,a_{in}|C_k)P(C_k)$ for all k in $\{1,2,\dots,K\}$ and select k corresponding to the maximal probability; it will indicate the class to which the data instance most probably belongs
- **Simplifying assumption: the attributes are independent** (this is why the method is called “naive”)
- $P(C_k|D_i)=P(a_{i1}|C_k)P(a_{i2}|C_k)\dots P(a_{in}|C_k)P(C_k)$
- This requires the knowledge of $P(a_{i1}|C_k)$, $P(a_{i2}|C_k)$, ..., $P(a_{in}|C_k)$ and $P(C_k)$
- **These probabilities can be estimated from the training dataset** (as relative frequencies) – this computations correspond to the learning process of Naïve Bayes

Naïve Bayes Classifier - example

Example:

Relation: weather.symbolic					
No.	outlook Nominal	temperature Nominal	humidity Nominal	windy Nominal	play Nominal
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no

$$P(C1)=P(\text{no})=5/14 \quad P(C2)=P(\text{yes})=9/14$$

A1: outlook

$$P(\text{sunny}|C1)=P(\text{sunny},C1)/P(C1) \\ = (3/14)/(5/14)=3/5$$

$$P(\text{sunny}|C2)=P(\text{sunny},C2)/P(C2) \\ = (2/14)/(9/14)=2/9$$

$$P(\text{overcast}|C1)=P(\text{overcast},C1)/P(C1) \\ = 0$$

$$P(\text{overcast}|C2)=P(\text{overcast},C2)/P(C2) \\ = (4/14)/(9/14)=4/9$$

$$P(\text{rainy}|C1)=P(\text{rainy},C1)/P(C1) \\ = (2/14)/(5/14)=2/5$$

$$P(\text{rainy}|C2)=P(\text{rainy},C2)/P(C2) \\ = (3/14)/(9/14)=3/9$$

Naïve Bayes Classifier - example

Example:

Relation: weather.symbolic					
No.	outlook Nominal	temperature Nominal	humidity Nominal	windy Nominal	play Nominal
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no

$$P(C1)=P(\text{no})=5/14 \quad P(C2)=P(\text{yes})=9/14$$

A2: temperature

$$P(\text{hot}|C1)=P(\text{hot},C1)/P(C1)=2/5$$

$$P(\text{hot}|C2)=P(\text{hot},C2)/P(C2)=2/9$$

$$P(\text{mild}|C1)=P(\text{mild},C1)/P(C1)=2/5$$

$$P(\text{mild}|C2)=P(\text{mild},C2)/P(C2)=4/9$$

$$P(\text{cool}|C1)=P(\text{cool},C1)/P(C1) \\ = (2/14)/(5/14)=1/5$$

$$P(\text{cool}|C2)=P(\text{cool},C2)/P(C2)=2/9$$

Naïve Bayes Classifier - example

Example:

Relation: weather.symbolic					
No.	outlook Nominal	temperature Nominal	humidity Nominal	windy Nominal	play Nominal
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no

$$P(C1)=P(\text{no})=5/14 \quad P(C2)=P(\text{yes})=9/14$$

A3: humidity

$$P(\text{high}|C1)=P(\text{high},C1)/P(C1)=4/5$$

$$P(\text{high}|C2)=P(\text{high},C2)/P(C2)=3/9$$

$$P(\text{normal}|C1)=P(\text{normal},C1)/P(C1)=1/5$$

$$P(\text{normal}|C2)=P(\text{normal},C2)/P(C2)=6/9$$

Naïve Bayes Classifier - example

Example:

Relation: weather.symbolic					
No.	outlook Nominal	temperature Nominal	humidity Nominal	windy Nominal	play Nominal
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no

$$P(C1)=P(\text{no})=5/14 \quad P(C2)=P(\text{yes})=9/14$$

A4: windy

$$P(\text{FALSE}|C1)=P(\text{FALSE},C1)/P(C1)=2/5$$

$$P(\text{FALSE}|C2)=P(\text{FALSE},C2)/P(C2)=6/9$$

$$P(\text{TRUE}|C1)=P(\text{TRUE},C1)/P(C1)=3/5$$

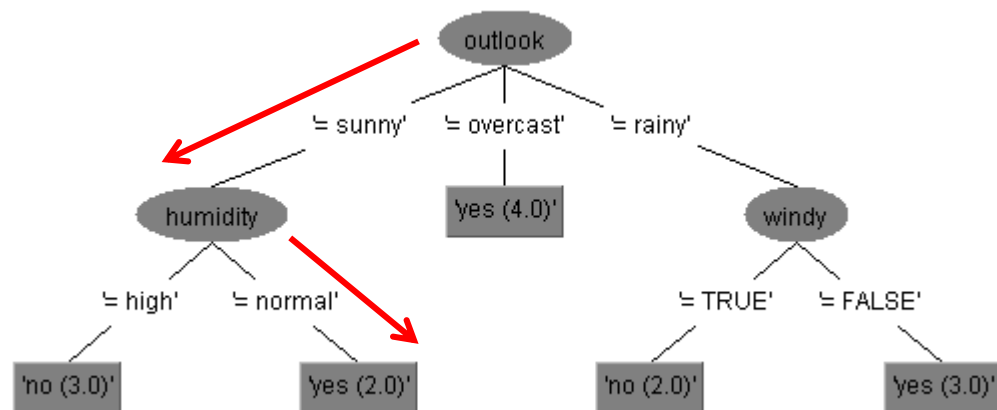
$$P(\text{TRUE}|C2)=P(\text{TRUE},C2)/P(C2)=3/9$$

Naïve Bayes Classifier - example

Example:

Relation: weather.symbolic

No.	outlook Nominal	temperature Nominal	humidity Nominal	windy Nominal	play Nominal
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no



$D=(\text{outlook}=\text{sunny}, \text{temperature}=\text{mild}, \text{humidity}=\text{normal}, \text{windy}=\text{False})$

$$P(C1|D)=P(\text{sunny}|C1)*P(\text{mild}|C1)*P(\text{normal}|C1)*P(\text{FALSE}|C1)*P(C1)/P(D)=$$

$$=3/5*2/5*1/5*2/5*5/14/P(D)=6/125/(14*P(D))=0.048/(14*P(D))$$

$$P(C2|D)=P(\text{sunny}|C2)*P(\text{mild}|C2)*P(\text{normal}|C2)*P(\text{FALSE}|C2)*P(C2)/P(D)=$$

$$=2/9*4/9*6/9*6/9*9/14/P(D)=144/729/(14*P(D))=0.197/(14*P(D)) \rightarrow \text{yes}$$

Naïve Bayes Classifier

Example:

Relation: weather.symbolic

No.	outlook Nominal	temperature Nominal	humidity Nominal	windy Nominal	play Nominal
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no

Remark: if for a given attribute value a_{ij} and a given class C_k there is no example in the training set, then $P(a_{ij} | C_k) = 0$ and (because of the independency assumption) for any instance having the value a_{ij} for attribute A_i the probability to belong to C_k will be 0.

- This situation might appear especially in the case of small classes
- In order to avoid such situations we can use the so-called Laplace smoothing

Laplace smoothing:

$$P(a_{ij} | C_k) = (\text{count}(a_{ij}, C_k) + \alpha) / (\text{count}(C_k) + m_i * \alpha)$$

α = Laplace smoothing parameter (small positive value, e.g. 1)

m_i = number of distinct values of attribute A_i

Naïve Bayes Classifier

Remarks:

- This classifier can be directly applied for **discrete attributes** and it is based on the following probabilistic models:
 - **Bernoulli** model (in the case of binary attributes)
 - **Multinoulli** model (in the case of discrete attributes with multiple values)
- In the case of attributes with values in continuous ranges there are two main approaches:
 - The attributes are discretized before using the classifier (the classifier performance depends on the discretization process)
 - The attributes are modeled through continuous probabilistic models (e.g. **Gaussian**) with parameters (e.g. mean and standard deviation) estimated based on the training data

Bayesian Network Classifiers

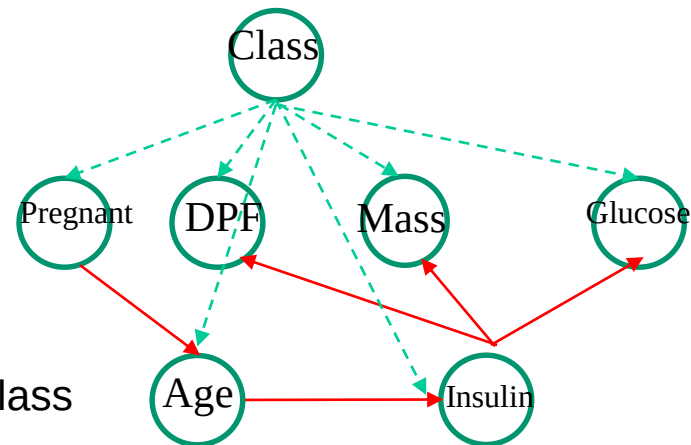
Motivation:

- The independence assumption of Naïve Bayes might be too restrictive
 - the Naive Bayes corresponds to a **one level tree-like structure** (root=class attribute, leaves = predictive attributes)
 - The interactions between attributes can be modelled through Bayesian networks (acyclic directed graph):
 - Node = attribute
 - Arc = interaction between attributes

Example (Pima dataset)

[Friedman et al, Bayesian Network Classifiers, 1997]

- Naïve Bayes:
 - Green dashed arrows
(correlation between the class attribute and the predictive attributes)
- Bayesian network:
 - Red arrows
(an arc between two nodes A and B means that the influence of node B on the class depends on node A)



Rmk: influence of **Insulin** value on the class depends on the value of **Age**

Bayesian Network Classifiers

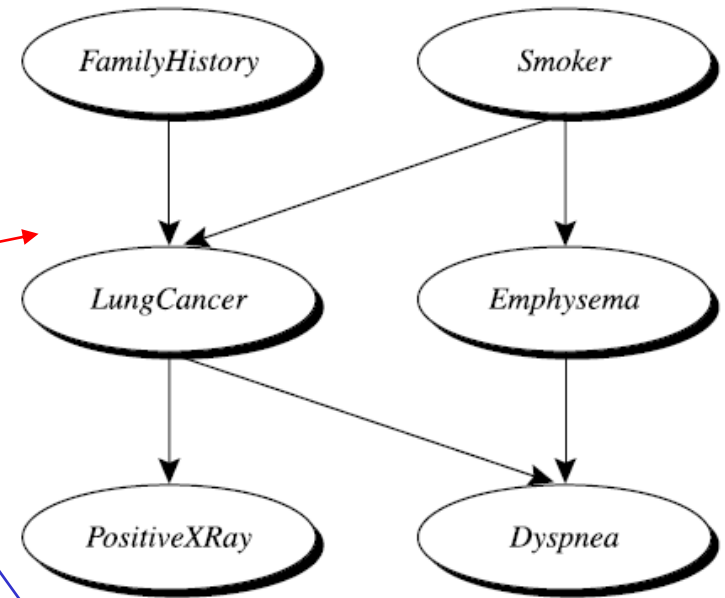
Example: lung cancer

- Binary random variables
- Components of the model
 - Topology: **causal network**
 - **Conditional Probability Tables**
- Main assumption: the variables are independent of their non-descendants given the parents

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{parents}(X_i))$$

Computation of probability of lung cancer given the other variables:

$$P(LC, FH, S, E, XR, D) = P(LC | FH, S) * P(FH) * P(S) * P(E | S) * P(XR | LC) * P(D | LC, E)$$



	<i>FH, S</i>	<i>FH, ~S</i>	<i>~FH, S</i>	<i>~FH, ~S</i>
<i>LC</i>	0.8	0.5	0.7	0.1
<i>~LC</i>	0.2	0.5	0.3	0.9

[Han et al., Text Mining.
Concepts and Techniques, 2023]

Bayesian Network Classifiers

- **Tree Augmented Naive Bayes** (TAN) – [Friedman et al, Bayesian Network Classifiers, 1997]
- **Idea**
 - Compute the Mutual Information between any two attributes and use it as weight for the edge between the corresponding nodes (in the complete graph)
 - Compute the maximum weight spanning tree (undirected)
 - Transform the undirected tree in a directed one by choosing the root (class attribute) and setting the direction of all edges to be outward of it
- **Implementations**
 - R – bnlearn (<https://www.bnlearn.com/examples/classifiers/>)
 - Python – pyAgrum (<https://pyagrum.readthedocs.io/en/latest/skbnClassifier.html>)

Summary – Bayesian models

- Advantages
 - It can be effective if the attributes are (nearly) independent (given the class)
 - Both the training and the classification steps are rather fast
- Disadvantages:
 - Naive Bayes ignores the interactions between attributes
 - When the attributes are not (conditionally) independent other Bayesian models can be used, e.g. Bayesian Networks (= graph-like models where nodes correspond to attributes and edges to relations between them)

Next lecture

- Function based classifiers
 - Neural Networks
 - Support Vector Machines